Technology Stack

Travel Itinerary and Expense Organizer

Current build & proposed deployment architecture

Database: `travel_itinerary_db`

Date: 26 October 2026

Document: Tech stack reference, version 2

> This document has two parts. **Part A** describes the database stack, which is built and verified. **Part B** describes the application layer (Python/FastAPI) and the Netlify/Vercel deployment. The API is implemented and tested; the hosting plan is still a plan. Read the box in Part B before choosing a host: **neither Netlify nor Vercel can host a MySQL database.**

Part A: the stack that is built (verified)

Everything in this part is in the repository today and was executed and tested on a live server.

Layer	Technology	Why / evidence
Database server	**MySQL 8.4 LTS** (minimum 8.0.16)	8.0 reached end of life in April 2026, so 8.4 LTS is the supported release. 8.0.16 is the floor because CHECK constraints are only enforced from that version. Verified on **8.4.13**.
Language	**SQL** (MySQL dialect)	All six scripts are `.sql`. Covers DDL, DML, DQL, TCL and stored programs.
Storage engine	**InnoDB**	`ENGINE=InnoDB` on all 32 tables. Required for foreign keys, transactions and row locking.
Character set	**utf8mb4** / `utf8mb4_0900_ai_ci`	Full Unicode, safe for names and symbols.
Client / IDE	**MySQL Workbench**	Runs the scripts, shows result grids, generates the EER diagram.
Modelling	**ER diagram** (21 entities)	Script 01 implements the approved ER diagram.
Version control	**Git + GitHub**	Repository `EPIC-555/DBMS-PROJECT`.

Database feature set in use

Construct	Count	Purpose
FOREIGN KEY	41	Referential integrity (CASCADE, RESTRICT, SET NULL)
CHECK constraint	34	Domain rules (rating 1-5, dates, positive money)
CREATE TRIGGER	31	24 BEFORE rules + 7 AFTER audit/alert triggers
CREATE VIEW	4	Derived data (nights, totals, dashboards, timeline)
CREATE PROCEDURE	2	`sp_create_trip`, `sp_book_room`
SIGNAL SQLSTATE	44	Business-rule rejections
UNIQUE	21	Uniqueness (email, confirmation code, seats)
ENUM	23	Fixed status/role lists
DECIMAL(10,2)	15	Exact money arithmetic (never FLOAT)
START TRANSACTION	6	Atomic multi-step operations
SUM() OVER (...)	1	Window function for a running total
SELECT ... FOR UPDATE	1	Row lock in `sp_book_room`

Verified 26 Oct 2026 on MySQL 8.4.13. Object counts match at every stage (01 → 21/27/12/1, 03 → 30/40/24/2, 05 → 32/41/31/4), all scripts are idempotent, all 24 rule tests across the demo scripts are rejected as documented, and both procedures run.

Part B: application and deployment stack

The API in section B.1 is **implemented and tested** (16 end-to-end tests against MySQL 8.4.13). The hosting choices in B.3 onward are still a plan.

B.1 Backend: Python (implemented)

Component	Choice and reason
Language	**Python 3.12** (Vercel's default; 3.13 and 3.14 also supported). Verified locally on 3.13.
Web framework	**FastAPI** — first-class on Vercel, async, automatic OpenAPI docs. Flask and Django are also supported.
MySQL driver	**mysql-connector-python** (official) or **PyMySQL** + SQLAlchemy for an ORM layer
Validation	**Pydantic** (ships with FastAPI)
Config	**python-dotenv** locally; environment variables in the host dashboard
Tests	**pytest** — 16 end-to-end tests, run against a real MySQL server, no mocks

The backend stays thin. All business rules live in the database as triggers, CHECK constraints, views and stored procedures, so the API mainly calls those and shapes the JSON. This keeps the single source of truth in MySQL and avoids duplicating validation. The trip recommendation itself is SQL: script `07_recommendation_engine.sql` adds the views `v_destination_stats`, `v_user_interest_profile` and `v_trip_suggestions` plus the procedure `sp_suggest_trips`, which score each unvisited destination out of 100 from ratings, wishlists, the user's own history and the travel season.

B.2 Frontend (optional)

- **Streamlit** — fastest route to a demo dashboard; can be deployed separately.
- **React + Vite —** if a custom UI is wanted; hosts cleanly on Netlify or Vercel as static files.

B.3 Hosting: the important constraint

> **Netlify and Vercel are serverless/static platforms. Neither one hosts a MySQL server.** A MySQL database is a long-running process with persistent disk; serverless functions are short-lived and stateless. You therefore need **two** services: a host for the code and a separate managed MySQL instance. This is normal and expected, not a problem with the plan.

Host	Python support	Notes
Vercel	**Yes, first-class.** ASGI/WSGI apps; FastAPI, Flask, Django detected automatically. Python 3.12 default.	Best fit for the Python backend. Databases are provisioned through the Vercel Marketplace, not as a native MySQL product.
Netlify	**Not for functions.** Netlify Functions run JavaScript/ TypeScript and Go. Python is available in the build environment only.	Great for the frontend and for Node/Go APIs. For a Python backend, use Vercel, Render, Railway or Fly.io instead.

B.4 Where the MySQL database should live

Option	Cost	Comment
Railway or **Aiven**	Free tier	Managed MySQL, connection string in one variable. Easiest path.
PlanetScale	Free tier	MySQL-compatible, but it is Vitess-based, so triggers and stored procedures are not fully supported. **Avoid for this project** — the design depends on 31 triggers.
Self-hosted VPS	Paid	Full control and full MySQL. More setup and maintenance.

Netlify's own database product is **Postgres**, and Vercel's Marketplace databases are mostly Postgres too. Neither is MySQL, so they would require rewriting the schema and all 31 triggers. Keep MySQL.

B.5 Reference architecture

Tier	Service	Role
Frontend	Netlify or Vercel (static)	Serves the UI (React/Vite or Streamlit)
Backend API	**Vercel** (Python runtime, FastAPI)	REST endpoints; calls views and stored procedures
Database	**Railway / Aiven** (managed MySQL)	Holds all 32 tables, 31 triggers, 4 views, 2 procedures
Secrets	Host environment variables	DB host, port, user, password, database name

B.6 Deployment steps

1. Provision a managed MySQL instance; run `01`, `03`, `05` against it.
2. Add the connection settings as environment variables in the host dashboard (never commit them).
3. Write the FastAPI app with an endpoint per query/view and one per stored procedure.
4. Push to GitHub; connect the repository to Vercel so each push becomes a preview deployment.
5. Point the frontend at the API base URL; deploy the frontend to Netlify or Vercel.
6. Optionally set a Vercel protection bypass secret if preview deployments are protected and automated testing is needed.

B.7 Points to be honest about

- The Python backend and hosting are **not implemented**; they are a proposal.
- The current password storage uses `SHA2(...,256)` for demo data only. A real API must use **bcrypt or argon2** with a salt.
- Serverless functions open a new database connection per invocation. Use a connection pool or a driver that supports it, or the managed MySQL instance will exhaust its connection limit under load.
- Cold starts add latency to the first request after idle; acceptable for a coursework demo.
- Keep credentials out of the repository and rotate any that were exposed.

Summary

Question	Answer
What is built today?	Pure SQL: MySQL 8.4 LTS, InnoDB, utf8mb4, 32 tables, 31 triggers, 4 views, 2 procedures. Verified on 8.4.13.
What is proposed?	A Python 3.12 + FastAPI backend, deployed on Vercel, with a static frontend, talking to a separately hosted managed MySQL database.
Can Netlify host the backend?	Not in Python. Netlify Functions are JavaScript/TypeScript and Go. Use Netlify for the frontend, or move the backend to Vercel.
Can either host MySQL?	No. Use Railway or Aiven for managed MySQL and keep the schema and triggers intact.